

# AI Beauty Makeup Analyzer - Complete MVP Documentation

## Table of Contents

- 1. [Project Overview](#)
  - 2. [Architecture & Design Patterns](#)
  - 3. [Technology Stack](#)
  - 4. [Database Schema](#)
  - 5. [Backend Implementation](#)
  - 6. [Frontend Implementation](#)
  - 7. [API Documentation](#)
  - 8. [Setup & Installation](#)
  - 9. [Development Guidelines](#)
- 

## Project Overview

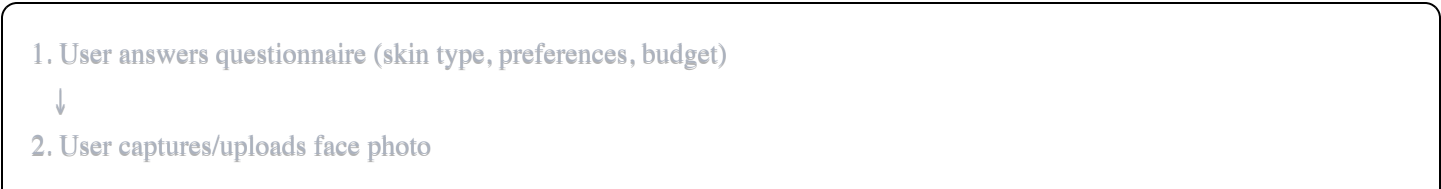
### Product Vision

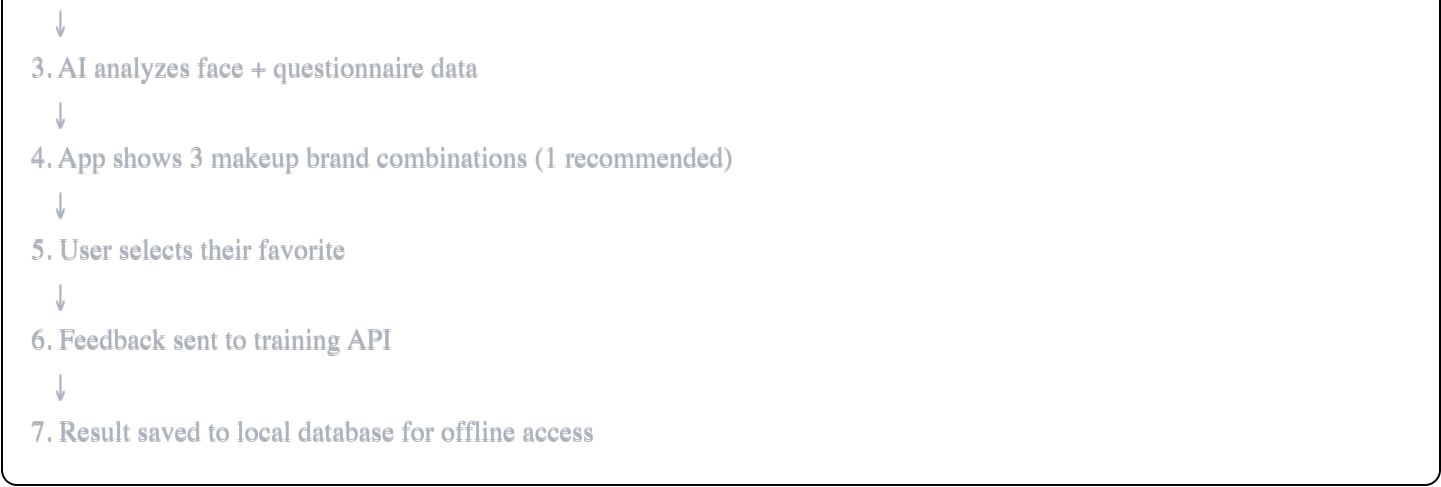
An AI-powered mobile application that helps users discover the perfect makeup products and application techniques based on their unique facial features, skin tone, and personal preferences. The app provides personalized recommendations from real beauty brands with continuous learning from user feedback.

### Core Features

- 1. **Interactive Questionnaire** - Skin type, tone, preferences, concerns
- 2. **AI Face Analysis** - Facial feature detection and analysis
- 3. **Brand-Based Recommendations** - Top 3 makeup combinations from real brands
- 4. **Save & Track Results** - Offline-accessible history of analyses
- 5. **Feedback Loop** - User ratings to improve AI recommendations
- 6. **Offline Support** - View saved analyses without internet

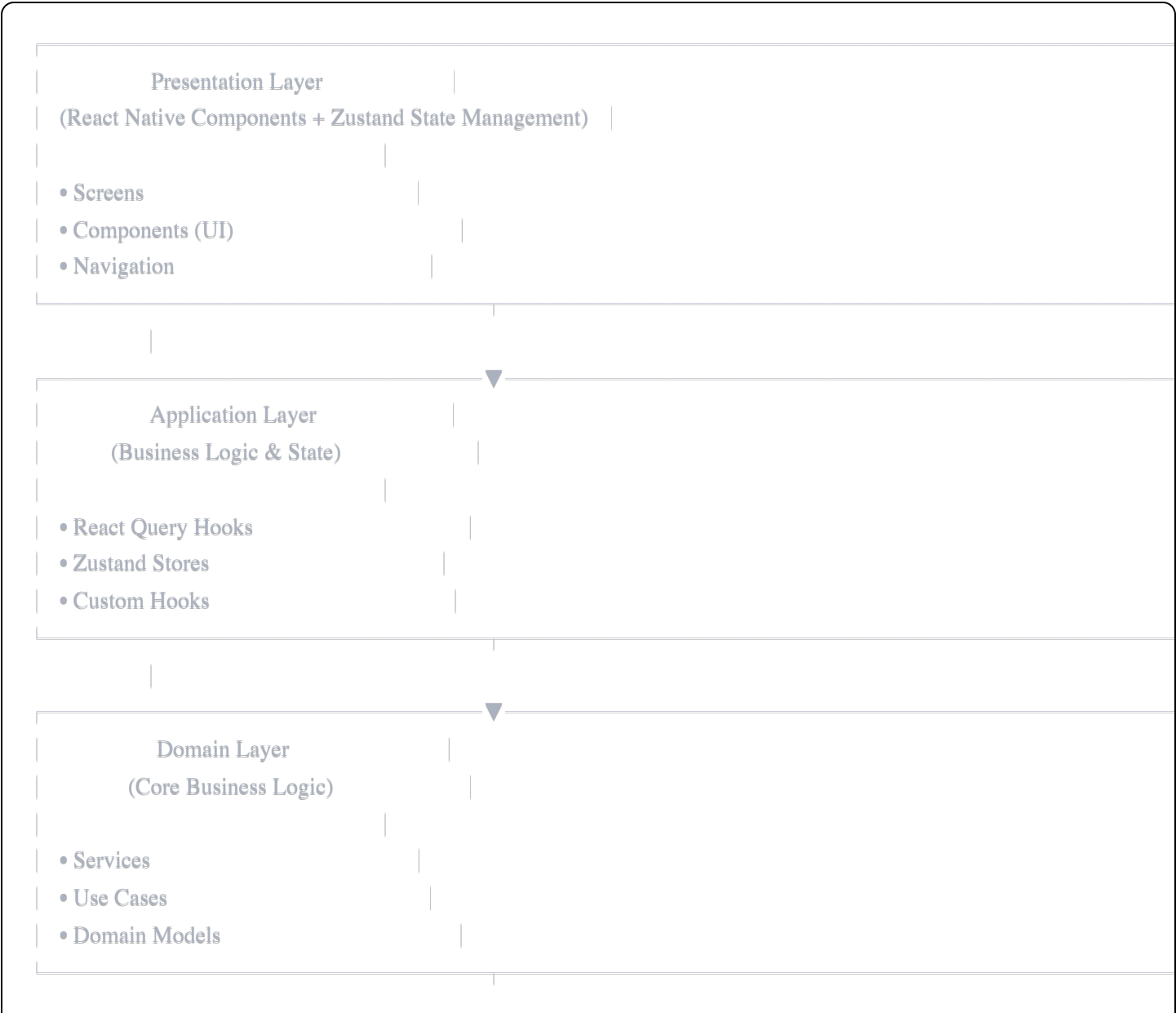
### User Flow





## Architecture & Design Patterns

### Clean Architecture Layers



## Infrastructure Layer (External Services & Data Access)

- API Client (Axios)
- Local Database (SQLite)
- File Storage (Cloudflare R2)

## Design Patterns Used

1. **Repository Pattern** - Data access abstraction
2. **Service Layer Pattern** - Business logic encapsulation
3. **Factory Pattern** - Object creation for recommendations
4. **Observer Pattern** - React Query for state management
5. **Singleton Pattern** - Database and API client instances
6. **Strategy Pattern** - Different analysis strategies
7. **Command Pattern** - User actions and API calls

## Technology Stack

### Backend

json

```
{
  "runtime": "Node.js 18+",
  "framework": "Express.js 4.18+",
  "database": "PostgreSQL 15+",
  "orm": "Prisma 5.0+",
  "fileStorage": "Cloudflare R2",
  "aiApi": "Anthropic Claude API",
  "validation": "Zod",
  "auth": "JWT (future)",
  "imageProcessing": "Sharp"
}
```

### Frontend (Mobile)

json

```
{  
  "framework": "React Native 0.74+ (Expo 51)",  
  "stateManagement": "Zustand 4.5+",  
  "dataFetching": "React Query (TanStack Query) 5.0+",  
  "httpClient": "Axios 1.6+",  
  "navigation": "React Navigation 6+",  
  "forms": "React Hook Form 7.5+",  
  "validation": "Zod",  
  "localDatabase": "Expo SQLite",  
  "styling": "NativeWind (Tailwind for RN)",  
  "icons": "Expo Icons"  
}
```

## Infrastructure

json

```
{  
  "hosting": "Railway / Render",  
  "database": "Neon PostgreSQL / Supabase",  
  "fileStorage": "Cloudflare R2",  
  "cdn": "Cloudflare CDN",  
  "monitoring": "Sentry (future)"  
}
```

---

## Database Schema

### PostgreSQL Schema (Prisma)

prisma

```
// schema.prisma

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

// User model (for future authentication)
model User {
  id          String    @id @default(cuid())
  email       String?    @unique
  deviceId    String     @unique
  createdAt   DateTime   @default(now())
  updatedAt   DateTime   @updatedAt

  analyses    Analysis[]
  feedbacks   Feedback[]

  @@index([deviceId])
}

// Analysis model - stores each face analysis session
model Analysis {
  id          String    @id @default(cuid())
  userId      String
  user        User      @relation(fields: [userId], references: [id], onDelete: Cascade)
}

// Questionnaire Data
skinType      String    // oily, dry, combination, normal, sensitive
skinTone      String    // fair, light, medium, tan, deep, rich
skinConcerns  String[]  // acne, aging, dark_spots, redness, etc.
makeupLevel   String    // beginner, intermediate, advanced
budget        String    // drugstore, mid_range, luxury, mixed
preferences   Json      // Additional preferences as JSON

// Image Data
imageUrl      String
imageCloudflareId String
```

```

// Face Analysis Results
faceShape      String
faceShapeConfidence Float
facialFeatures  Json    // Structured facial feature data
landmarks      Json    // Facial landmark coordinates


// Recommendations
recommendations Recommendation[]
recommendedIndex Int     @default(0) // Index of the recommended option (0-2)


// Metadata
analysisVersion String  @default("1.0")
processingTime  Int?    // milliseconds
createdAt      DateTime @default(now())
updatedAt      DateTime @updatedAt

feedbacks      Feedback[]

@@index([userId, createdAt])
@@index([createdAt])
}

// Recommendation model - stores each makeup combination recommendation
model Recommendation {
  id          String  @id @default(cuid())
  analysisId  String
  analysis    Analysis @relation(fields: [analysisId], references: [id], onDelete: Cascade)

  rank        Int     // 1, 2, or 3
  isRecommended Boolean @default(false)


// Product Information
foundation  Json    // { brand, product, shade, price, link }
concealer   Json
powder      Json
blush       Json
bronzer     Json
highlighter Json
eyebrowProduct Json
eyeshadow   Json
eyeliner    Json
mascara     Json
lipProduct  Json

```

```

// Application Guidance
applicationTips String[]
techniques      Json      // Step-by-step techniques

// Reasoning
whyRecommended String // AI explanation for this combination
matchScore      Float   @default(0.0) // 0-100

// Metadata
totalPrice      Float?
createdAt        DateTime @default(now())

@@index([analysisId, rank])
}

// Feedback model - user feedback for training
model Feedback {
    id          String @id @default(cuid())
    userId       String
    user         User   @relation(fields: [userId], references: [id], onDelete: Cascade)
    analysisId   String
    analysis     Analysis @relation(fields: [analysisId], references: [id], onDelete: Cascade)

// User Selection
selectedRank    Int      // Which recommendation user chose (1-3)
selectedRecommendation String // ID reference for faster queries

// Ratings (1-5 scale)
overallRating   Int?
accuracyRating  Int?    // How well it matched their needs
productQualityRating Int?

// Detailed Feedback
whatWorked      String[] // Aspects they liked
whatDidntWork   String[] // Aspects they didn't like
additionalNotes String?

// Training Data Flags
usedForTraining Boolean @default(false)
trainingBatchId String?

// Metadata
createdAt        DateTime @default(now())

```

```

    @@index([analysisId])
    @@index([usedForTraining, createdAt])
}

// Brand Product Catalog (for reference and matching)
model Product {
    id          String    @id @default(cuid())

    brand       String
    productName String
    category     String    // foundation, concealer, blush, etc.
    subcategory  String?   // liquid, powder, cream, etc.

    // Product Details
    shades       Json[]    // Array of available shades
    price         Float
    priceRange   String    // drugstore, mid_range, luxury

    // Matching Criteria
    skinTypes     String[] // Which skin types it's good for
    skinTones     String[] // Which tones it matches
    coverage      String?  // sheer, medium, full
    finish        String?  // matte, dewy, satin, natural

    // Links & Images
    productUrl   String?
    imageUrl     String?

    // Metadata
    rating        Float?
    reviewCount   Int?
    isActive      Boolean @default(true)
    createdAt     DateTime @default(now())
    updatedAt     DateTime @updatedAt

    @@index([category, brand])
    @@index([skinTypes, skinTones])
}

// Training Dataset (aggregated for AI improvement)
model TrainingData {
    id          String    @id @default(cuid())

    // Input Features

```



```
skinType      String
skinTone      String
faceShape     String
facialFeatures Json
skinConcerns  String[]
budget        String
```

```
// Output Labels (what worked)
```

```
successfulProducts Json // Products user selected
```

```
rating        Float
```

```
// Metadata
```

```
feedbackCount Int    @default(1)
```

```
lastUpdated    DateTime @updatedAt
```

```
createdAt      DateTime @default(now())
```

```
@@index([skinType, skinTone, faceShape])
```

```
}
```

## SQLite Schema (Mobile Local Storage)

```
sql
```

-- Local database for offline access

-- Using Expo SQLite

```
CREATE TABLE IF NOT EXISTS analyses (  
  id TEXT PRIMARY KEY,  
  skin_type TEXT NOT NULL,  
  skin_tone TEXT NOT NULL,  
  skin_concerns TEXT, -- JSON array as string  
  makeup_level TEXT,  
  budget TEXT,  
  image_uri TEXT,  
  image_cloudflare_url TEXT,  
  face_shape TEXT,  
  facial_features TEXT, -- JSON  
  recommendations TEXT, -- JSON array  
  recommended_index INTEGER DEFAULT 0,  
  created_at INTEGER NOT NULL,  
  synced BOOLEAN DEFAULT 0  
);
```

```
CREATE TABLE IF NOT EXISTS feedback (  
  id TEXT PRIMARY KEY,  
  analysis_id TEXT NOT NULL,  
  selected_rank INTEGER NOT NULL,  
  overall_rating INTEGER,  
  notes TEXT,  
  created_at INTEGER NOT NULL,  
  synced BOOLEAN DEFAULT 0,  
  FOREIGN KEY (analysis_id) REFERENCES analyses(id)  
);
```

```
CREATE INDEX idx_analyses_created ON analyses(created_at DESC);
```

```
CREATE INDEX idx_analyses_synced ON analyses(synced);
```

```
CREATE INDEX idx_feedback_analysis ON feedback(analysis_id);
```

## Backend Implementation

### Project Structure

backend/

```
|—— src/
|   |—— config/
|   |   |—— database.ts
|   |   |—— cloudflare.ts
|   |   |—— anthropic.ts
|   |—— controllers/
|   |   |—— analysis.controller.ts
|   |   |—— feedback.controller.ts
|   |   |—— product.controller.ts
|   |—— services/
|   |   |—— analysis.service.ts
|   |   |—— claudeservice.ts
|   |   |—— recommendation.service.ts
|   |   |—— storage.service.ts
|   |   |—— training.service.ts
|   |—— repositories/
|   |   |—— analysis.repository.ts
|   |   |—— feedback.repository.ts
|   |   |—— product.repository.ts
|   |—— models/
|   |   |—— analysis.model.ts
|   |   |—— recommendation.model.ts
|   |—— validators/
|   |   |—— analysis.validator.ts
|   |   |—— feedback.validator.ts
|   |—— middlewares/
|   |   |—— error.middleware.ts
|   |   |—— validation.middleware.ts
|   |   |—— upload.middleware.ts
|   |—— routes/
|   |   |—— analysis.routes.ts
|   |   |—— feedback.routes.ts
|   |   |—— index.ts
|   |—— utils/
|   |   |—— logger.ts
|   |   |—— response.ts
|   |   |—— constants.ts
|   |—— types/
|   |   |—— index.ts
|   |—— app.ts
|—— prisma/
|   |—— schema.prisma
|   |—— migrations/
|   |—— seed.ts
```

- └── .env.example
- └── .gitignore
- └── package.json
- └── tsconfig.json
- └── README.md

## Core Files

### 1. Configuration Files

**src/config/database.ts**

typescript

```

import { PrismaClient } from '@prisma/client';
import { logger } from '../utils/logger';

class DatabaseService {
  private static instance: PrismaClient;

  private constructor() {}

  public static getInstance(): PrismaClient {
    if (!DatabaseService.instance) {
      DatabaseService.instance = new PrismaClient({
        log: process.env.NODE_ENV === 'development'
          ? ['query', 'error', 'warn']
          : ['error'],
      });

      DatabaseService.instance.$connect()
        .then(() => logger.info('Database connected successfully'))
        .catch((error) => logger.error('Database connection error:', error));
    }

    return DatabaseService.instance;
  }

  public static async disconnect(): Promise<void> {
    if (DatabaseService.instance) {
      await DatabaseService.instance.$disconnect();
      logger.info('Database disconnected');
    }
  }
}

export const prisma = DatabaseService.getInstance();

```

## src/config/cloudflare.ts

typescript

```

import { S3Client } from '@aws-sdk/client-s3';
import { Upload } from '@aws-sdk/lib-storage';
import { v4 as uuidv4 } from 'uuid';

export class CloudflareR2Service {
  private static instance: CloudflareR2Service;
  private s3Client: S3Client;
  private bucketName: string;
  private publicUrl: string;

  private constructor() {
    this.s3Client = new S3Client({
      region: 'auto',
      endpoint: process.env.CLOUDFLARE_R2_ENDPOINT!,
      credentials: {
        accessKeyId: process.env.CLOUDFLARE_R2_ACCESS_KEY_ID!,
        secretAccessKey: process.env.CLOUDFLARE_R2_SECRET_ACCESS_KEY!,
      },
    });

    this.bucketName = process.env.CLOUDFLARE_R2_BUCKET_NAME!;
    this.publicUrl = process.env.CLOUDFLARE_R2_PUBLIC_URL!;
  }

  public static getInstance(): CloudflareR2Service {
    if (!CloudflareR2Service.instance) {
      CloudflareR2Service.instance = new CloudflareR2Service();
    }
    return CloudflareR2Service.instance;
  }

  async uploadImage(
    buffer: Buffer,
    contentType: string
  ): Promise<{ url: string; cloudflareId: string }> {
    const cloudflareId = `${uuidv4()}.jpg`;
    const key = `analyses/${new Date().getFullYear()}/${cloudflareId}`;

    const upload = new Upload({
      client: this.s3Client,
      params: {
        Bucket: this.bucketName,
        Key: key,

```

```
    Body: buffer,
    ContentType: contentType,
    CacheControl: 'max-age=31536000',
  },
});

await upload.done();

return {
  url: `${this.publicUrl}/${key}`,
  cloudflareId: key,
};
}

getPublicUrl(cloudflareId: string): string {
  return `${this.publicUrl}/${cloudflareId}`;
}
}

export const cloudflareStorage = CloudflareR2Service.getInstance();
```

## src/config/anthropic.ts

typescript

```
import Anthropic from '@anthropic-ai/sdk';

class ClaudeService {
  private static instance: Anthropic;

  private constructor() {}

  public static getInstance(): Anthropic {
    if (!ClaudeService.instance) {
      ClaudeService.instance = new Anthropic({
        apiKey: process.env.ANTHROPIC_API_KEY!,
      });
    }

    return ClaudeService.instance;
  }
}

export const anthropic = ClaudeService.getInstance();
```

## 2. Type Definitions

**src/types/index.ts**

typescript



```

import { z } from 'zod';

// Questionnaire Schema
export const QuestionnaireSchema = z.object({
  skinType: z.enum(['oily', 'dry', 'combination', 'normal', 'sensitive']),
  skinTone: z.enum(['fair', 'light', 'medium', 'tan', 'deep', 'rich']),
  skinConcerns: z.array(
    z.enum([
      'acne',
      'aging',
      'dark_spots',
      'redness',
      'large_pores',
      'dryness',
      'oiliness',
      'sensitivity',
      'uneven_tone',
    ])
  ),
  makeupLevel: z.enum(['beginner', 'intermediate', 'advanced']),
  budget: z.enum(['drugstore', 'mid_range', 'luxury', 'mixed']),
  preferences: z.object({
    preferredBrands: z.array(z.string()).optional(),
    avoidIngredients: z.array(z.string()).optional(),
    isVegan: z.boolean().optional(),
    isCrueltyFree: z.boolean().optional(),
    preferredFinish: z.enum(['matte', 'dewy', 'natural', 'satin']).optional(),
    coverage: z.enum(['sheer', 'light', 'medium', 'full']).optional(),
  }).optional(),
});

export type Questionnaire = z.infer<typeof QuestionnaireSchema>;

// Analysis Request Schema
export const AnalysisRequestSchema = z.object({
  deviceId: z.string(),
  questionnaire: QuestionnaireSchema,
});

export type AnalysisRequest = z.infer<typeof AnalysisRequestSchema>;

// Feedback Schema
export const FeedbackSchema = z.object({

```

```
analysisId: z.string(),
selectedRank: z.number().int().min(1).max(3),
overallRating: z.number().int().min(1).max(5).optional(),
accuracyRating: z.number().int().min(1).max(5).optional(),
productQualityRating: z.number().int().min(1).max(5).optional(),
whatWorked: z.array(z.string()).optional(),
whatDidntWork: z.array(z.string()).optional(),
additionalNotes: z.string().optional(),
});
```

```
export type FeedbackInput = z.infer<typeof FeedbackSchema>;
```

```
// Product Interface
```

```
export interface Product {
  brand: string;
  productName: string;
  shade: string;
  price: number;
  link: string;
  imageUrl?: string;
}
```

```
// Recommendation Interface
```

```
export interface RecommendationData {
  rank: number;
  isRecommended: boolean;
  foundation: Product;
  concealer: Product;
  powder: Product;
  blush: Product;
  bronzer: Product;
  highlighter: Product;
  eyebrowProduct: Product;
  eyeshadow: Product;
  eyeliner: Product;
  mascara: Product;
  lipProduct: Product;
  applicationTips: string[];
  techniques: {
    step: number;
    area: string;
    instruction: string;
    product: string;
  }[];
};
```

```
    whyRecommended: string;
    matchScore: number;
    totalPrice: number;
}
```

*// Claude Analysis Response*

```
export interface ClaudeAnalysisResponse {
    faceShape: string;
    faceShapeConfidence: number;
    facialFeatures: {
        foreheadSize: string;
        cheekboneProminence: string;
        jawlineDefinition: string;
        noseShape: string;
        lipFullness: string;
        eyeShape: string;
        eyebrowShape: string;
    };
    landmarks: {
        [key: string]: { x: number; y: number };
    };
    skinAnalysis: {
        undertone: string;
        textureNotes: string;
        visibleConcerns: string[];
    };
    recommendations: RecommendationData[];
    recommendedIndex: number;
}
```

*// API Response Types*

```
export interface ApiResponse<T = any> {
    success: boolean;
    data?: T;
    error?: string;
    message?: string;
    timestamp: string;
}
```

```
export interface PaginatedResponse<T> extends ApiResponse<T> {
    pagination: {
        page: number;
        limit: number;
        total: number;
    };
}
```

```
    totalPages: number;
  };
}
```

### 3. Validators

#### src/validators/analysis.validator.ts

typescript

```
import { z } from 'zod';
import { QuestionnaireSchema, AnalysisRequestSchema } from '../types';

export class AnalysisValidator {
  static validateQuestionnaire(data: unknown) {
    return QuestionnaireSchema.parse(data);
  }

  static validateAnalysisRequest(data: unknown) {
    return AnalysisRequestSchema.parse(data);
  }

  static validateImage(file: Express.Multer.File): void {
    const allowedMimeTypes = ['image/jpeg', 'image/png', 'image/jpg'];
    const maxSize = 10 * 1024 * 1024; // 10MB

    if (!file) {
      throw new Error('No image file provided');
    }

    if (!allowedMimeTypes.includes(file.mimetype)) {
      throw new Error('Invalid image format. Only JPEG and PNG allowed');
    }

    if (file.size > maxSize) {
      throw new Error('Image size exceeds 10MB limit');
    }
  }
}
```

#### src/validators/feedback.validator.ts

typescript

```
import { FeedbackSchema } from '../types';

export class FeedbackValidator {
  static validateFeedback(data: unknown) {
    return FeedbackSchema.parse(data);
  }
}
```

## 4. Services

**src/services/claude.service.ts**

typescript

```
import { anthropic } from '../config/anthropic';
import { Questionnaire, ClaudeAnalysisResponse } from '../types';
import { logger } from '../utils/logger';

export class ClaudeAnalysisService {
  private static readonly MODEL = 'claude-sonnet-4-20250514';
  private static readonly MAX_TOKENS = 4096;

  static async analyzeImage(
    base64Image: string,
    questionnaire: Questionnaire
  ): Promise<ClaudeAnalysisResponse> {
    const prompt = this.buildAnalysisPrompt(questionnaire);

    try {
      const response = await anthropic.messages.create({
        model: this.MODEL,
        max_tokens: this.MAX_TOKENS,
        messages: [
          {
            role: 'user',
            content: [
              {
                type: 'image',
                source: {
                  type: 'base64',
                  media_type: 'image/jpeg',
                  data: base64Image,
                },
              },
              {
                type: 'text',
                text: prompt,
              },
            ],
          },
        ],
      });

      const textContent = response.content.find(
        (block) => block.type === 'text'
      );
    }
  }
}
```

```

if (!textContent || textContent.type !== 'text') {
    throw new Error('No text response from Claude');
}

return this.parseClaudeResponse(textContent.text);
} catch (error) {
    logger.error('Claude API error:', error);
    throw new Error('Failed to analyze image with AI');
}
}

private static buildAnalysisPrompt(questionnaire: Questionnaire): string {
    return `You are an expert makeup artist and beauty consultant. Analyze this person's face and provide personalized makeup

**User Profile:**
- Skin Type: ${questionnaire.skinType}
- Skin Tone: ${questionnaire.skinTone}
- Skin Concerns: ${questionnaire.skinConcerns.join(', ')}
- Makeup Level: ${questionnaire.makeupLevel}
- Budget: ${questionnaire.budget}
${questionnaire.preferences ? ` - Preferences: ${JSON.stringify(questionnaire.preferences, null, 2)} ` : ""}

**Task:**
1. Analyze facial features (face shape, bone structure, features)
2. Detect skin undertone and texture
3. Provide 3 complete makeup product combinations from REAL brands
4. Recommend the BEST option (mark as recommended)
5. Include detailed application techniques

**Response Format (JSON only, no markdown):**
{
  "faceShape": "ovalroundlsquarelheartldiamondloblong",
  "faceShapeConfidence": 0.0-1.0,
  "facialFeatures": {
    "foreheadSize": "smalllmediumllarge",
    "cheekboneProminence": "lowlmediumlhigh",
    "jawlineDefinition": "softlmoderatedefined",
    "noseShape": "description",
    "lipFullness": "thinlmediumlfull",
    "eyeShape": "almondroundlhoodedlmonolidlturnedldownturned",
    "eyebrowShape": "straightlarchedsoft_archlrounded"
  },
  "landmarks": {
    "foreheadCenter": {"x": 0-100, "y": 0-100},

```

```
"cheekbonesLeft": {"x": 0-100, "y": 0-100},
"cheekbonesRight": {"x": 0-100, "y": 0-100},
"chinCenter": {"x": 0-100, "y": 0-100},
"noseBridge": {"x": 0-100, "y": 0-100}
},
"skinAnalysis": {
  "undertone": "warm|cool|neutral",
  "textureNotes": "description of visible texture",
  "visibleConcerns": ["concern1", "concern2"]
},
"recommendations": [
  {
    "rank": 1,
    "isRecommended": true,
    "foundation": {
      "brand": "Fenty Beauty",
      "productName": "Pro Filt'r Soft Matte Foundation",
      "shade": "290",
      "price": 40.00,
      "link": "https://example.com/product",
      "imageUrl": "https://example.com/image.jpg"
    },
    "concealer": { /* same structure */ },
    "powder": { /* same structure */ },
    "blush": { /* same structure */ },
    "bronzer": { /* same structure */ },
    "highlighter": { /* same structure */ },
    "eyebrowProduct": { /* same structure */ },
    "eyeshadow": { /* same structure */ },
    "eyeliner": { /* same structure */ },
    "mascara": { /* same structure */ },
    "lipProduct": { /* same structure */ },
    "applicationTips": [
      "tip 1",
      "tip 2"
    ],
    "techniques": [
      {
        "step": 1,
        "area": "face",
        "instruction": "detailed instruction",
        "product": "foundation"
      }
    ]
  },
  {
    "rank": 2,
    "isRecommended": false,
    "foundation": {
      "brand": "Glossier",
      "productName": "Cloud Paint",
      "shade": "pink",
      "price": 15.00,
      "link": "https://example.com/product",
      "imageUrl": "https://example.com/image.jpg"
    },
    "concealer": { /* same structure */ },
    "powder": { /* same structure */ },
    "blush": { /* same structure */ },
    "bronzer": { /* same structure */ },
    "highlighter": { /* same structure */ },
    "eyebrowProduct": { /* same structure */ },
    "eyeshadow": { /* same structure */ },
    "eyeliner": { /* same structure */ },
    "mascara": { /* same structure */ },
    "lipProduct": { /* same structure */ },
    "applicationTips": [
      "tip 1",
      "tip 2"
    ],
    "techniques": [
      {
        "step": 1,
        "area": "face",
        "instruction": "detailed instruction",
        "product": "foundation"
      }
    ]
  }
]
```



```

    "whyRecommended": "Detailed explanation why this is the best match",
    "matchScore": 95.0,
    "totalPrice": 450.00
  },
  {
    "rank": 2,
    "isRecommended": false,
    /* ... second option ... */
  },
  {
    "rank": 3,
    "isRecommended": false,
    /* ... third option ... */
  }
],
"recommendedIndex": 0
}

```

\*\*\*Important Guidelines:\*\*\*

- Use REAL makeup brands (Fenty, MAC, NARS, Maybelline, NYX, etc.)
- Match products to user's budget preference
- Provide actual shade names/numbers
- Ensure products work well together
- Consider skin type and concerns in product selection
- Make one recommendation clearly the best match
- Be specific with application techniques
- Price accuracy is important

Return ONLY valid JSON, no markdown formatting or code blocks.`;

```

}
```

```

private static parseClaudeResponse(text: string): ClaudeAnalysisResponse {
  // Remove markdown code blocks if present
  const jsonMatch = text.match(/```json\n?([\s\S]*?)\n?```/) || text.match(/\s{[\s\S]*\s}/);

  if (!jsonMatch) {
    throw new Error('Could not find JSON in Claude response');
  }

  const jsonStr = jsonMatch[1] || jsonMatch[0];

  try {
    return JSON.parse(jsonStr);
  } catch (error) {

```

```
    logger.error('JSON parse error:', error);  
    throw new Error('Invalid JSON response from AI');  
  }  
}  
}
```

## src/services/storage.service.ts

typescript

```

import sharp from 'sharp';
import { cloudflareStorage } from '../config/cloudflare';
import { logger } from '../utils/logger';

export class StorageService {
  static async processAndUploadImage(
    buffer: Buffer
  ): Promise<{ url: string; cloudflareId: string }> {
    try {
      // Process image with sharp
      const processedBuffer = await sharp(buffer)
        .resize(1024, 1024, {
          fit: 'inside',
          withoutEnlargement: true,
        })
        .jpeg({ quality: 85 })
        .toBuffer();

      // Upload to Cloudflare R2
      const result = await cloudflareStorage.uploadImage(
        processedBuffer,
        'image/jpeg'
      );

      logger.info('Image uploaded successfully:', result.cloudflareId);
      return result;
    } catch (error) {
      logger.error('Image processing error:', error);
      throw new Error('Failed to process and upload image');
    }
  }

  static getImageUrl(cloudflareId: string): string {
    return cloudflareStorage.getPublicUrl(cloudflareId);
  }
}

```

src/services/recommendation.service.ts

typescript

```

import { prisma } from '../config/database';
import { RecommendationData } from '../types';
import { logger } from '../utils/logger';

export class RecommendationService {
  static async saveRecommendations(
    analysisId: string,
    recommendations: RecommendationData[]
  ): Promise<void> {
    try {
      await prisma.recommendation.createMany({
        data: recommendations.map((rec) => ({
          analysisId,
          rank: rec.rank,
          isRecommended: rec.isRecommended,
          foundation: rec.foundation,
          concealer: rec.concealer,
          powder: rec.powder,
          blush: rec.blush,
          bronzer: rec.bronzer,
          highlighter: rec.highlighter,
          eyebrowProduct: rec.eyebrowProduct,
          eyeshadow: rec.eyeshadow,
          eyeliner: rec.eyeliner,
          mascara: rec.mascara,
          lipProduct: rec.lipProduct,
          applicationTips: rec.applicationTips,
          techniques: rec.techniques,
          whyRecommended: rec.whyRecommended,
          matchScore: rec.matchScore,
          totalPrice: rec.totalPrice,
        })),
      });

      logger.info(`Saved ${recommendations.length} recommendations for analysis ${analysisId}`);
    } catch (error) {
      logger.error('Error saving recommendations:', error);
      throw error;
    }
  }
}

```



```
import { prisma } from './config/database';
import { ClaudeAnalysisService } from './claude.service';
import { StorageService } from './storage.service';
import { RecommendationService } from './recommendation.service';
import { Questionnaire, ClaudeAnalysisResponse } from './types';
import { logger } from './utils/logger';

export class AnalysisService {
  static async createAnalysis(
    deviceId: string,
    questionnaire: Questionnaire,
    imageBuffer: Buffer
  ) {
    const startTime = Date.now();

    try {
      // 1. Ensure user exists
      const user = await this.ensureUser(deviceId);

      // 2. Process and upload image
      const { url: imageUrl, cloudflareId } = await StorageService.processAndUploadImage(
        imageBuffer
      );

      // 3. Convert image to base64 for Claude
      const base64Image = imageBuffer.toString('base64');

      // 4. Analyze with Claude
      const aiAnalysis: ClaudeAnalysisResponse = await ClaudeAnalysisService.analyzeImage(
        base64Image,
        questionnaire
      );

      // 5. Create analysis record
      const analysis = await prisma.analysis.create({
        data: {
          userId: user.id,
          skinType: questionnaire.skinType,
          skinTone: questionnaire.skinTone,
          skinConcerns: questionnaire.skinConcerns,
          makeupLevel: questionnaire.makeupLevel,
          budget: questionnaire.budget,
          preferences: questionnaire.preferences || {},
        },
      });
    } catch (error) {
      logger.error('Error creating analysis: ', error);
    }
  }

  private async ensureUser(deviceId: string): Promise<User> {
    let user = await prisma.user.findFirst({
      where: { deviceId },
    });

    if (!user) {
      user = await prisma.user.create({
        data: { deviceId },
      });
    }

    return user;
  }
}
```

```
    imageUrl,  
    imageCloudflareId: cloudflareId,  
    faceShape: aiAnalysis.faceShape,  
    faceShapeConfidence: aiAnalysis.faceShapeConfidence,  
    facialFeatures: aiAnalysis.facialFeatures,  
    landmarks: aiAnalysis.landmarks,  
    recommendedIndex: aiAnalysis.recommendedIndex,  
    processingTime: Date.now() - startTime,  
  },  
});
```

*// 6. Save recommendations*

```
await RecommendationService.saveRecommendations(  
  analysis.id,  
  aiAnalysis.recommendations  
);
```

*// 7. Fetch complete analysis with recommendations*

```
const completeAnalysis = await prisma.analysis.findUnique({  
  where: { id: analysis.id },  
  include: {  
    recommendations: {  
      orderBy: { rank: 'asc' },  
    },  
  },  
});
```

```
logger.info(`Analysis completed in ${Date.now() - startTime}ms`);
```

```
  return completeAnalysis;  
} catch (error) {  
  logger.error('Analysis service error:', error);  
  throw error;  
}  
}
```

```
private static async ensureUser(deviceId: string) {  
  let user = await prisma.user.findUnique({  
    where: { deviceId },  
  });
```

```
  if (!user) {  
    user = await prisma.user.create({  
      data: { deviceId },
```

```
});  
logger.info(`Created new user: ${user.id}`);  
}
```

```
return user;  
}
```

```
static async getUserAnalyses(  
  deviceId: string,  
  page: number = 1,  
  limit: number = 20  
) {  
  const user = await prisma.user.findUnique({  
    where: { deviceId },  
  });
```

```
  if (!user) {  
    return { analyses: [], total: 0 };  
  }
```

```
  const [analyses, total] = await Promise.all([  
    prisma.analysis.findMany({  
      where: { userId: user.id },  
      include: {  
        recommendations: {  
          orderBy: { rank: 'asc' },  
        },  
      },  
      orderBy: { createdAt: 'desc' },  
      skip: (page - 1) * limit,  
      take: limit,  
    }),  
    prisma.analysis.count({  
      where: { userId: user.id },  
    }),  
  ]);
```

```
  return { analyses, total };  
}
```

```
static async getAnalysisById(analysisId: string) {  
  return await prisma.analysis.findUnique({  
    where: { id: analysisId },  
    include: {
```



```
recommendations: {  
  orderBy: { rank: 'asc' },  
},  
},  
});  
}  
}
```

## src/services/training.service.ts

typescript

```
import { prisma } from '../config/database';
import { logger } from '../utils/logger';

export class TrainingService {
  static async submitFeedbackForTraining(feedbackId: string) {
    try {
      const feedback = await prisma.feedback.findUnique({
        where: { id: feedbackId },
        include: {
          analysis: {
            include: {
              recommendations: true,
            },
          },
        },
      });

      if (!feedback) {
        throw new Error('Feedback not found');
      }

      // Get the selected recommendation
      const selectedRec = feedback.analysis.recommendations.find(
        (r) => r.rank === feedback.selectedRank
      );

      if (!selectedRec) {
        throw new Error('Selected recommendation not found');
      }

      // Create or update training data
      const trainingDataKey = `${feedback.analysis.skinType}-${feedback.analysis.skinTone}-${feedback.analysis.faceShape}`

      const existingData = await prisma.trainingData.findFirst({
        where: {
          skinType: feedback.analysis.skinType,
          skinTone: feedback.analysis.skinTone,
          faceShape: feedback.analysis.faceShape,
        },
      });

      const successfulProducts = {
        foundation: selectedRec.foundation,
```

```
concealer: selectedRec.concealer,  
powder: selectedRec.powder,  
blush: selectedRec.blush,  
bronzer: selectedRec.bronzer,  
highlighter: selectedRec.highlighter,  
eyebrowProduct: selectedRec.eyebrowProduct,  
eyeshadow: selectedRec.eyeshadow,  
eyeliner: selectedRec.eyeliner,  
mascara: selectedRec.mascara,  
lipProduct: selectedRec.lipProduct,  
};
```

```
if (existingData) {  
  await prisma.trainingData.update({  
    where: { id: existingData.id },  
    data: {  
      rating: (existingData.rating * existingData.feedbackCount + (feedback.overallRating || 0)) / (existingData.feedbackCount + 1),  
      feedbackCount: existingData.feedbackCount + 1,  
    },  
  });  
} else {  
  await prisma.trainingData.create({  
    data: {  
      skinType: feedback.analysis.skinType,  
      skinTone: feedback.analysis.skinTone,  
      faceShape: feedback.analysis.faceShape,  
      facialFeatures: feedback.analysis.facialFeatures,  
      skinConcerns: feedback.analysis.skinConcerns,  
      budget: feedback.analysis.budget,  
      successfulProducts:  
      rating: feedback.overallRating || 0,  
    },  
  });  
}
```

```
// Mark feedback as used for training
```

```
await prisma.feedback.update({  
  where: { id: feedbackId },  
  data: { usedForTraining: true },  
});
```

```
logger.info(`Feedback ${feedbackId} submitted for training`);  
} catch (error) {  
  logger.error('Error submitting feedback for training:', error);  
}
```

```
throw error;
```

```
}
```

```
}
```

```
}
```

## 5. Repositories

**src/repositories/feedback.repository.ts**

typescript

```
import { prisma } from './config/database';
import { FeedbackInput } from './types';

export class FeedbackRepository {
  static async create(deviceId: string, feedbackData: FeedbackInput) {
    // Get user
    const user = await prisma.user.findUnique({
      where: { deviceId },
    });

    if (!user) {
      throw new Error('User not found');
    }

    // Get analysis and recommendation
    const analysis = await prisma.analysis.findUnique({
      where: { id: feedbackData.analysisId },
      include: { recommendations: true },
    });

    if (!analysis) {
      throw new Error('Analysis not found');
    }

    const selectedRecommendation = analysis.recommendations.find(
      (r) => r.rank === feedbackData.selectedRank
    );

    if (!selectedRecommendation) {
      throw new Error('Selected recommendation not found');
    }

    // Create feedback
    return await prisma.feedback.create({
      data: {
        userId: user.id,
        analysisId: feedbackData.analysisId,
        selectedRank: feedbackData.selectedRank,
        selectedRecommendation: selectedRecommendation.id,
        overallRating: feedbackData.overallRating,
        accuracyRating: feedbackData.accuracyRating,
        productQualityRating: feedbackData.productQualityRating,
        whatWorked: feedbackData.whatWorked || [],
      },
    });
  }
}
```

```
    whatDidntWork: feedbackData.whatDidntWork || [],  
    additionalNotes: feedbackData.additionalNotes,  
  },  
});  
}
```

```
static async findById(id: string) {  
  return await prisma.feedback.findUnique({  
    where: { id },  
    include: {  
      analysis: {  
        include: {  
          recommendations: true,  
        },  
      },  
    },  
  });  
}
```

## 6. Controllers

**src/controllers/analysis.controller.ts**

typescript

```

import { Request, Response, NextFunction } from 'express';
import { AnalysisService } from '../services/analysis.service';
import { AnalysisValidator } from '../validators/analysis.validator';
import { successResponse, errorResponse } from '../utils/response';
import { logger } from '../utils/logger';

export class AnalysisController {
  static async createAnalysis(req: Request, res: Response, next: NextFunction) {
    try {
      // Validate image
      if (!req.file) {
        return errorResponse(res, 'No image file provided', 400);
      }

      AnalysisValidator.validateImage(req.file);

      // Validate request body
      const { deviceId, questionnaire } = AnalysisValidator.validateAnalysisRequest({
        deviceId: req.body.deviceId,
        questionnaire: JSON.parse(req.body.questionnaire),
      });

      // Create analysis
      const analysis = await AnalysisService.createAnalysis(
        deviceId,
        questionnaire,
        req.file.buffer
      );

      return successResponse(res, analysis, 'Analysis completed successfully', 201);
    } catch (error) {
      logger.error('Create analysis error:', error);
      next(error);
    }
  }

  static async getUserAnalyses(req: Request, res: Response, next: NextFunction) {
    try {
      const deviceId = req.query.deviceId as string;
      const page = parseInt(req.query.page as string) || 1;
      const limit = parseInt(req.query.limit as string) || 20;

      if (!deviceId) {

```

```
    return ErrorResponse(res, 'Device ID is required', 400);
  }

  const { analyses, total } = await AnalysisService.getUserAnalyses(
    deviceId,
    page,
    limit
  );

  return successResponse(
    res,
    {
      analyses,
      pagination: {
        page,
        limit,
        total,
        totalPages: Math.ceil(total / limit),
      },
    },
    'Analyses retrieved successfully'
  );
} catch (error) {
  logger.error('Get user analyses error:', error);
  next(error);
}
}

static async getAnalysisById(req: Request, res: Response, next: NextFunction) {
  try {
    const { id } = req.params;

    const analysis = await AnalysisService.getAnalysisById(id);

    if (!analysis) {
      return ErrorResponse(res, 'Analysis not found', 404);
    }

    return successResponse(res, analysis, 'Analysis retrieved successfully');
  } catch (error) {
    logger.error('Get analysis by ID error:', error);
    next(error);
  }
}
```



```
}
```

```
}
```

**src/controllers/feedback.controller.ts**

typescript

```

import { Request, Response, NextFunction } from 'express';
import { FeedbackRepository } from '../repositories/feedback.repository';
import { TrainingService } from '../services/training.service';
import { FeedbackValidator } from '../validators/feedback.validator';
import { successResponse, errorResponse } from '../utils/response';
import { logger } from '../utils/logger';

export class FeedbackController {
  static async submitFeedback(req: Request, res: Response, next: NextFunction) {
    try {
      const deviceId = req.body.deviceId as string;

      if (!deviceId) {
        return errorResponse(res, 'Device ID is required', 400);
      }

      // Validate feedback data
      const feedbackData = FeedbackValidator.validateFeedback(req.body);

      // Create feedback
      const feedback = await FeedbackRepository.create(deviceId, feedbackData);

      // Submit for training (async, don't wait)
      TrainingService.submitFeedbackForTraining(feedback.id).catch((error) => {
        logger.error('Training submission error:', error);
      });

      return successResponse(
        res,
        feedback,
        'Feedback submitted successfully. Thank you for helping us improve!',
        201
      );
    } catch (error) {
      logger.error('Submit feedback error:', error);
      next(error);
    }
  }

  static async getFeedbackById(req: Request, res: Response, next: NextFunction) {
    try {
      const { id } = req.params;

```

```

const feedback = await FeedbackRepository.findById(id);

if (!feedback) {
  return errorResponse(res, 'Feedback not found', 404);
}

return successResponse(res, feedback, 'Feedback retrieved successfully');
} catch (error) {
  logger.error('Get feedback error:', error);
  next(error);
}
}
}

```

## 7. Routes

### src/routes/analysis.routes.ts

```

typescript

import { Router } from 'express';
import multer from 'multer';
import { AnalysisController } from '../controllers/analysis.controller';

const router = Router();
const upload = multer({
  storage: multer.memoryStorage(),
  limits: { fileSize: 10 * 1024 * 1024 }, // 10MB
});

router.post(
  '/analyze',
  upload.single('image'),
  AnalysisController.createAnalysis
);

router.get('/', AnalysisController.getUserAnalyses);

router.get('/:id', AnalysisController.getAnalysisById);

export default router;

```

### src/routes/feedback.routes.ts

typescript

```
import { Router } from 'express';
import { FeedbackController } from '../controllers/feedback.controller';

const router = Router();

router.post('/', FeedbackController.submitFeedback);

router.get('/:id', FeedbackController.getFeedbackById);

export default router;
```

## src/routes/index.ts

typescript

```
import { Router } from 'express';
import analysisRoutes from './analysis.routes';
import feedbackRoutes from './feedback.routes';

const router = Router();

router.use('/analysis', analysisRoutes);
router.use('/feedback', feedbackRoutes);

// Health check
router.get('/health', (req, res) => {
  res.json({
    success: true,
    message: 'API is healthy',
    timestamp: new Date().toISOString(),
  });
});

export default router;
```

## 8. Middlewares

### src/middlewares/error.middleware.ts

typescript

```

import { Request, Response, NextFunction } from 'express';
import { ZodError } from 'zod';
import { logger } from '../utils/logger';
import { errorResponse } from '../utils/response';

export function errorHandler(
  error: Error,
  req: Request,
  res: Response,
  next: NextFunction
) {
  logger.error('Error:', error);

  // Zod validation errors
  if (error instanceof ZodError) {
    return errorResponse(
      res,
      'Validation error',
      400,
      error.errors.map((e) => ({
        field: e.path.join('.'),
        message: e.message,
      })))
    );
  }

  // Default error
  const statusCode = (error as any).statusCode || 500;
  const message = error.message || 'Internal server error';

  return errorResponse(res, message, statusCode);
}

export function notFoundHandler(req: Request, res: Response) {
  return errorResponse(res, `Route ${req.url} not found`, 404);
}

```

## 9. Utilities

### src/utils/response.ts

typescript

```
import { Response } from 'express';
import { ApiResponse } from '../types';

export function successResponse<T>(
  res: Response,
  data: T,
  message: string = 'Success',
  statusCode: number = 200
): Response<ApiResponse<T>> {
  return res.status(statusCode).json({
    success: true,
    data,
    message,
    timestamp: new Date().toISOString(),
  });
}

export function errorResponse(
  res: Response,
  message: string,
  statusCode: number = 500,
  errors?: any
): Response<ApiResponse> {
  return res.status(statusCode).json({
    success: false,
    error: message,
    ...(errors && { errors }),
    timestamp: new Date().toISOString(),
  });
}
```

**src/utils/logger.ts**

typescript

```
export const logger = {  
  info: (...args: any[]) => {  
    console.log('[INFO]', new Date().toISOString(), ...args);  
  },  
  error: (...args: any[]) => {  
    console.error('[ERROR]', new Date().toISOString(), ...args);  
  },  
  warn: (...args: any[]) => {  
    console.warn('[WARN]', new Date().toISOString(), ...args);  
  },  
  debug: (...args: any[]) => {  
    if (process.env.NODE_ENV === 'development') {  
      console.debug('[DEBUG]', new Date().toISOString(), ...args);  
    }  
  },  
};
```

## 10. Main Application

src/app.ts

typescript

```

import express from 'express';
import cors from 'cors';
import helmet from 'helmet';
import routes from './routes';
import { errorHandler, notFoundHandler } from './middlewares/error.middleware';

const app = express();

// Security middleware
app.use(helmet());
app.use(cors({
  origin: process.env.ALLOWED_ORIGINS?.split(',') || '*',
  credentials: true,
}));

// Body parsing
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// Routes
app.use('/api/v1', routes);

// Error handling
app.use(notFoundHandler);
app.use(errorHandler);

const PORT = process.env.PORT || 3000;

app.listen(PORT, () => {
  console.log( Server running on port ${PORT}`);
  console.log( Environment: ${process.env.NODE_ENV}`);
});

export default app;

```

## **.env.example**

```
env
```



#### # Server

NODE\_ENV=development

PORT=3000

ALLOWED\_ORIGINS=http://localhost:19000,http://localhost:19006

#### # Database

DATABASE\_URL=postgresql://user:password@localhost:5432/beauty\_ai

#### # Anthropic

ANTHROPIC\_API\_KEY=your\_anthropic\_api\_key

#### # Cloudflare R2

CLOUDFLARE\_R2\_ENDPOINT=https://your-account-id.r2.cloudflarestorage.com

CLOUDFLARE\_R2\_ACCESS\_KEY\_ID=your\_access\_key

CLOUDFLARE\_R2\_SECRET\_ACCESS\_KEY=your\_secret\_key

CLOUDFLARE\_R2\_BUCKET\_NAME=beauty-ai-images

CLOUDFLARE\_R2\_PUBLIC\_URL=https://your-bucket.r2.dev

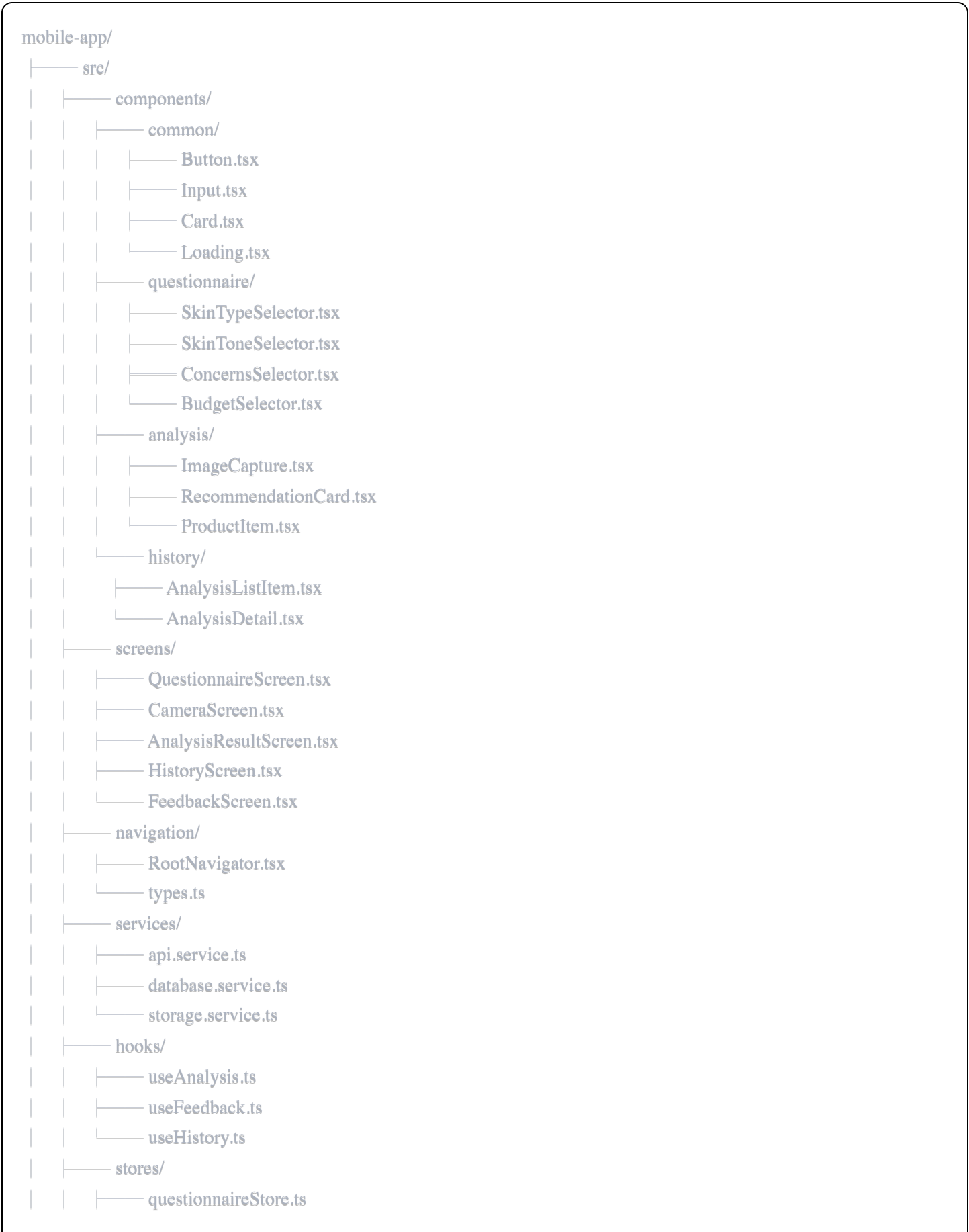
### package.json

json

```
{
  "name": "beauty-ai-backend",
  "version": "1.0.0",
  "scripts": {
    "dev": "ts-node-dev --respawn --transpile-only src/app.ts",
    "build": "tsc",
    "start": "node dist/app.js",
    "prisma:generate": "prisma generate",
    "prisma:migrate": "prisma migrate dev",
    "prisma:deploy": "prisma migrate deploy",
    "prisma:seed": "ts-node prisma/seed.ts"
  },
  "dependencies": {
    "@anthropic-ai/sdk": "^0.30.0",
    "@aws-sdk/client-s3": "^3.699.0",
    "@aws-sdk/lib-storage": "^3.699.0",
    "@prisma/client": "^5.22.0",
    "axios": "^1.7.9",
    "cors": "^2.8.5",
    "dotenv": "^16.4.7",
    "express": "^4.21.2",
    "helmet": "^8.0.0",
    "multer": "^1.4.5-lts.1",
    "sharp": "^0.33.5",
    "uuid": "^11.0.4",
    "zod": "^3.24.1"
  },
  "devDependencies": {
    "@types/cors": "^2.8.17",
    "@types/express": "^5.0.0",
    "@types/multer": "^1.4.12",
    "@types/node": "^22.10.5",
    "@types/uuid": "^10.0.0",
    "prisma": "^5.22.0",
    "ts-node": "^10.9.2",
    "ts-node-dev": "^2.0.0",
    "typescript": "^5.7.3"
  }
}
```

# Frontend Implementation

## Project Structure



```
graph LR; Root[ ] --- analysisStore.ts[analysisStore.ts]; Root --- historyStore.ts[historyStore.ts]; Root --- types[/types/]; Root --- index.ts[index.ts]; Root --- utils[/utils/]; Root --- constants.ts[constants.ts]; Root --- helpers.ts[helpers.ts]; Root --- App.tsx[App.tsx]; Root --- app.json[app.json]; Root --- package.json[package.json]; Root --- tsconfig.json[tsconfig.json]; types --- types_index.ts[types/index.ts];
```

- analysisStore.ts
- historyStore.ts
- types/
  - index.ts
- utils/
- constants.ts
- helpers.ts
- App.tsx
- app.json
- package.json
- tsconfig.json

This documentation continues in the next artifact...